

SuperBox: An Open Marketplace for Secure Discovery, Verification, and Sandboxed Execution of Model Context Protocol Servers

Areeb Ahmed Shariff

*Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
areebahmed0709@gmail.com*

Amartya Anand

*Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
amartyaanand07@gmail.com*

Arush Verma

*Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
arush3218@gmail.com*

Devansh Aryan

*Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
devansh123aryan@gmail.com*

Abstract—The rapid adoption of Model Context Protocol (MCP) servers has allowed AI applications to interact with external tools, databases, and data sources, enabling context-aware AI assistants and autonomous agents. However, the MCP ecosystem faces major challenges, including the lack of a central registry, inconsistent security checks, and no standardized setup. Servers are currently discovered manually through scattered GitHub repositories and run directly on developer machines, with no mechanism to assess reliability, verify security compliance, or manage deployments across cloud and agent environments. These gaps constrain enterprise adoption and expose users to real security risks when integrating third-party MCP servers.

SuperBox addresses these gaps as an open-source marketplace and serverless execution platform for MCP servers. It enforces an automated five-stage security pipeline comprising SonarCloud for static analysis, Snyk for dependency vulnerability detection, GitGuardian for secret exposure prevention, and Bandit for Python-specific security linting. SuperBox pairs a high-performance Go REST API with a developer-facing CLI for publishing, discovery, and client configuration. AWS Lambda and WebSocket API Gateway provide fully isolated, per-session execution environments with enforced memory limits, timeouts, and controlled network access, supporting real-time JSON-RPC communication.

By combining serverless execution, automated security validation, S3-backed discovery, and auto-generated client configuration, SuperBox reduces the operational complexity of deploying MCP servers while enforcing consistent security standards. The platform is validated through security pipeline evaluation, execution performance benchmarks, and end-to-end publish-to-execution cycle testing across representative MCP server configurations. SuperBox is released as an open-source platform, providing the MCP ecosystem with a validated, production-ready foundation for secure server discovery, verification, and execution.

Index Terms—Model Context Protocol, Serverless Computing, AWS Lambda, WebSocket API, Security Automation, AI Agent Infrastructure, Cloud-Native Architecture, JSON-RPC

I. INTRODUCTION

The Model Context Protocol (MCP), introduced by Anthropic in late 2024, defines a standardized bidirectional communication interface between large language models and external tools, APIs, and data sources [1]. Within its first year of release, the protocol surpassed eight million weekly SDK downloads, signaling rapid and widespread adoption across AI development teams [10]. MCP enables AI applications to move beyond static knowledge, allowing them to query live databases, invoke APIs, read file systems, and execute code through a unified protocol layer. This shift has made MCP servers a foundational component in the emerging ecosystem of autonomous AI agents and context-aware assistants.

Despite this growth, the MCP ecosystem lacks the infrastructure required for reliable, secure, and scalable server deployment. Developers currently discover MCP servers through scattered GitHub repositories, personal recommendations, and informal aggregator sites, with no central registry enforcing any form of quality or security standard. Most MCP servers are consumed directly from source with no scanning, no validation, and no isolation, meaning the code runs directly on the developer's machine with access to local resources. Empirical studies of the ecosystem have found that more than half of listed servers are non-functional or low-value [4], that dependency monocultures create systemic risks, and that malicious servers have been uploaded to widely-used aggregation platforms without triggering any audit mechanism [7]. This creates a critical trust gap that constrains enterprise adoption and exposes users to real security risks including data exfiltration, credential theft, and tool poisoning.

This paper presents SuperBox, an open-source marketplace and serverless execution platform for MCP servers. Taking

inspiration from the registry-and-validation model established by Docker Hub for container images, SuperBox adapts that paradigm to the MCP ecosystem and addresses the above challenges through three integrated contributions. First, a centralized S3-backed registry with enforced metadata standards provides a single, trusted source for MCP server discovery. Second, an automated five-stage security pipeline covering static analysis, dependency scanning, secret detection, and tool discovery ensures that only validated servers enter the registry. Third, a serverless execution layer built on AWS Lambda and WebSocket API Gateway provides completely isolated, on-demand sandboxed execution environments for every MCP server invocation. Together, these components eliminate the manual, fragmented, and insecure patterns that currently characterize MCP server integration.

The remainder of this paper is organized as follows: Section II reviews related work on MCP ecosystem security and deployment challenges. Section III describes the system design and architecture. Section IV covers the implementation details. Section V presents results and evaluation. Section VI concludes with future work directions.

II. RELATED WORK

The security and reliability challenges of the MCP ecosystem have attracted significant research attention since the protocol’s release. A comprehensive lifecycle analysis of MCP servers structures the problem across four phases (creation, deployment, operation, and maintenance) and identifies sixteen distinct threat scenarios across four attacker categories including malicious developers, external attackers, and security flaws inherent to the protocol itself [3]. This foundational taxonomy establishes that MCP’s open and extensible design, while powerful, introduces trust challenges at every phase of the server lifecycle that cannot be addressed through runtime defenses alone.

Large-scale empirical measurement of the ecosystem provides concrete evidence of the fragmentation problem. A systematic study aggregating over 8,400 valid MCP projects from six major markets found that more than half of listed projects are invalid or low-value, that servers exhibit structural risks including dependency monocultures and uneven maintenance, and that no centralized mechanism exists for enforcing quality or trust at the point of publication [4]. A parallel study of 1,899 open-source MCP servers using a hybrid static analysis pipeline found eight distinct vulnerability classes unique to MCP: 7.2% of servers contained general security vulnerabilities and 5.5% exhibited MCP-specific tool poisoning, neither detectable by traditional static analysis tools [6]. Pre-deployment scanning must therefore be MCP-aware to surface threats that general-purpose tools would miss.

On the attack surface, research has identified several novel threat classes specific to MCP’s architecture. Parasitic Toolchain Attacks exploit MCP’s lack of context-tool isolation by embedding malicious instructions in external data sources accessed during legitimate LLM tasks; the attack was demonstrated across a census of 12,230 tools from 1,360

real servers, enabling stealthy data exfiltration [5]. Preference Manipulation Attacks demonstrate that customized MCP servers can be crafted to influence LLMs into prioritizing them over competing servers, representing a commercially motivated threat that exploits open registration models [8]. Tool poisoning, shadowing, and rug pull attacks have been demonstrated across multiple model architectures; layered defenses combining manifest signing and semantic vetting show partial mitigation but at the cost of increased latency [3].

An end-to-end empirical attack evaluation confirmed that malicious MCP servers were successfully uploaded to three widely-used aggregation platforms without triggering any audit mechanism, and that users consistently failed to distinguish malicious servers from legitimate ones [7].

Proposed solutions in the literature primarily target runtime and protocol-level defenses. A Zero Trust registry architecture with admin-controlled registration, dynamic trust scoring based on versioning and known vulnerabilities, and just-in-time credential provisioning has been proposed to counter tool squatting in multi-agent systems [9]. However, no existing platform integrates pre-deployment security validation, centralized registry management, and isolated execution into a single deployable system. SuperBox addresses this gap by moving the security boundary upstream to the point of publication, combining pre-deployment validation with sandboxed execution to provide defense across the full server lifecycle.

III. SYSTEM DESIGN AND ARCHITECTURE

A. Overview

SuperBox is composed of three loosely coupled layers: the publish and validation layer operated through the developer CLI, the discovery and management layer served by the Go REST API and S3 registry, and the execution layer powered by AWS Lambda and WebSocket API Gateway. Each layer operates independently, communicates through well-defined interfaces, and can be scaled or replaced without affecting the others. This separation of concerns ensures that the security pipeline, registry management, and execution sandbox never share runtime state, minimizing the blast radius of any failure or compromise in a single layer.

B. Security Pipeline

The pipeline is designed against a threat model in which the primary adversary is a developer who intentionally or negligently publishes a server containing malicious code, exposed secrets, vulnerable dependencies, or tool poisoning payloads. Threats requiring runtime network interception or hardware-level compromise of the execution environment are considered out of scope.

When a developer initiates a publish workflow, the CLI sequentially executes five validation stages against the target GitHub repository before any data is written to the registry. SonarCloud performs static code analysis, detecting code

smells, bugs, security hotspots, and cognitive complexity issues across the full repository. Snyk audits all declared dependencies in `requirements.txt` and `package.json` files against its vulnerability database, identifying known CVEs and suggesting remediation paths. GitGuardian scans all files and commit history for hardcoded secrets, API keys, tokens, and credentials. Bandit performs Python-specific security linting, flagging dangerous function calls, insecure configurations, and injection-prone patterns. A regex-based tool discovery pass then extracts the names and signatures of all MCP tools exposed by the server from the source code.

Each stage produces a structured result object. If any stage reports a critical finding, the pipeline halts immediately and returns a consolidated report to the developer. Only repositories that clear all five stages proceed to registration. Table I summarizes the coverage of each stage.

TABLE I
SECURITY PIPELINE STAGE COVERAGE

Stage	Tool	Scope
Static Analysis	SonarCloud	Code smells, bugs, hotspots, complexity
Dependency Scan	Snyk	Known CVEs in declared dependencies
Secret Detection	GitGuardian	Hardcoded keys, tokens, credentials
Python Linting	Bandit	Dangerous calls, injection-prone patterns
Tool Discovery	Regex pass	MCP tool names, signatures, decorators

C. Registry Design

The registry is implemented as a flat-file store in an AWS S3 bucket, where each MCP server is represented as a single JSON object stored at the bucket root under the server’s name as the key. This design eliminates the operational overhead of a relational database for what is fundamentally a read-heavy workload, enables direct access from both the Go API and the Lambda executor without an intermediate data layer, and produces a registry format that is portable, auditable, and inspectable without any tooling. Each registry record contains the server name, repository reference, entrypoint file, runtime language, list of discovered tools, tool count, full security report from all five pipeline stages, pricing configuration, and creation and update timestamps. Registry contents are exposed through authenticated REST endpoints covering listing, search, retrieval, creation, update, and deletion. Table II lists all fields in each registry record.

D. Execution Layer

The execution layer bridges two fundamentally different communication models: the stdio-based JSON-RPC interface expected by MCP servers and the WebSocket-based interface used by AI clients connecting to the cloud. A lightweight local proxy process, started by the CLI during client configuration, reads JSON-RPC messages from the AI client’s stdin and forwards them over a persistent WebSocket connection to

TABLE II
S3 REGISTRY JSON RECORD STRUCTURE

Field	Description
<code>name</code>	Unique server identifier (registry key)
<code>repo</code>	GitHub repository URL
<code>entrypoint</code>	Entry file path within the repository
<code>language</code>	Runtime language (Python, Node.js)
<code>tools</code>	List of discovered MCP tool descriptors
<code>tool_count</code>	Total number of exposed tools
<code>security</code>	Full five-stage pipeline report object
<code>pricing</code>	Tier and price configuration (free/paid)
<code>created_at</code>	ISO 8601 timestamp of first publication
<code>updated_at</code>	ISO 8601 timestamp of last update

the AWS WebSocket API Gateway. The API Gateway routes the connection lifecycle events and messages to the Lambda function handler.

The Lambda handler exposes three WebSocket route handlers managing the session lifecycle: connection setup, per-message dispatch, and disconnection cleanup. On the first inbound message, the handler bootstraps a fresh execution environment, performs the MCP initialization handshake automatically, and then forwards the original request. All subsequent messages are forwarded directly. On disconnect, the server subprocess is terminated and all temporary files are purged, ensuring complete isolation between sessions.

E. Architecture Summary

The three layers interact through the following paths. The publish path flows from the developer CLI through the five-stage security pipeline into the S3 registry. The discovery path flows from web and API clients through the Go REST API, which reads directly from S3. The execution path flows from an AI client through a local proxy over WebSocket to the API Gateway, then to Lambda, which bootstraps the server process and streams responses back through the same connection.

IV. IMPLEMENTATION

A. CLI and Publish Workflow

The CLI is implemented in Python 3.11 using the Click framework and installed as a standalone package via pip. The CLI exposes nine commands: `auth` for registration, login, logout, status, and token refresh via Firebase; `init` to scaffold a new `superbox.json` configuration file; `push` to execute the full security pipeline and publish to the registry; `pull` to fetch registry metadata and write AI client configuration; `run` to start an interactive JSON-RPC test session against a deployed server; `search` to query the registry by name or keyword; `inspect` to display full server details and security report; `test` to run a server locally from a repository URL without registry; and `logs` to fetch and optionally stream CloudWatch execution logs.

The `push` command is the core of the publish workflow. It reads the local `superbox.json` configuration file to extract the repository URL, entrypoint, and language. It verifies Firebase authentication by checking the stored token

against the Identity Toolkit API. It then runs the five-stage security pipeline sequentially, cloning the repository to a temporary directory for local scanning stages. Tool discovery uses regex patterns to identify MCP tool decorator definitions across Python source files and `package.json` definitions for Node.js servers. On successful completion of all stages, the command serializes the full scan results and server metadata into a JSON record and uploads it to the S3 registry bucket using boto3.

Table III lists all nine commands.

TABLE III
CLI COMMAND REFERENCE

Command	Description
auth	Register, login, logout, token refresh via Firebase
init	Scaffold a new <code>superbox.json</code> config file
push	Run security pipeline and publish to registry
pull	Fetch server metadata and write client config
run	Start interactive JSON-RPC session against server
search	Query registry by name or keyword
inspect	Display full server details and security report
test	Run server locally from repo URL without registry
logs	Fetch and stream CloudWatch execution logs

B. API Server

The REST API is implemented in Go 1.26 using the Gin web framework. It exposes versioned endpoints under `/api/v1` for server management (GET `/servers`, GET `/servers/:name`, POST `/servers`, PUT `/servers/:name`, DELETE `/servers/:name`) and authentication (POST `/auth/register`, POST `/auth/login`, POST `/auth/device`, GET `/auth/profile`). Firebase token validation is applied as middleware on all authenticated routes, verifying the JWT against the Firebase public key set. S3 operations are delegated to a Python helper subprocess using boto3, which the Go server invokes via `exec.Command` and communicates with through `stdin/stdout`. CORS is enabled for all origins to support web client access. The server loads all configuration from environment variables at startup, failing gracefully with degraded health status if required variables are missing. Table IV summarizes the full API surface.

C. Lambda Executor

The execution handler is implemented in Python 3.11 and deployed to AWS Lambda with 1,024 MB memory and a 60-second timeout, configurable up to 900 seconds [2]. The handler maintains a module-level process reference and connection parameter store to persist the running MCP server process across multiple WebSocket messages within a single connection lifecycle. The `$connect` handler extracts and

TABLE IV
API ENDPOINT SUMMARY

Method	Endpoint	Auth	Purpose
GET	<code>/api/v1/servers</code>	No	List all registered servers
GET	<code>/api/v1/servers/:name</code>	No	Get server details and security report
POST	<code>/api/v1/servers</code>	Yes	Register a new server (CLI push)
PUT	<code>/api/v1/servers/:name</code>	Yes	Update an existing server record
DELETE	<code>/api/v1/servers/:name</code>	Yes	Remove a server from the registry
POST	<code>/api/v1/auth/register</code>	No	Create a new user account
POST	<code>/api/v1/auth/login</code>	No	Authenticate and receive JWT
POST	<code>/api/v1/auth/device</code>	No	Initiate CLI device flow OAuth
GET	<code>/api/v1/auth/profile</code>	Yes	Get authenticated user profile

stores the query string parameters including the server name and optional test mode flags. The `$default` handler checks whether the MCP process is running; if not, it fetches the server record from S3, clones the GitHub repository by downloading the ZIP archive and extracting it to `/tmp`, runs `pip install -r requirements.txt` with the target directory set to `/tmp/pip_modules`, and starts the MCP server as a subprocess with `stdio` pipes. It then performs the MCP initialize handshake and notifications/initialized notification before processing the original request. The `$disconnect` handler kills the subprocess and removes all temporary files. Responses are sent back to the connected client using the `apigatewaymanagementapi` boto3 client with the connection's domain name and stage reconstructed from the event context.

D. Frontend

The web marketplace is implemented in Next.js 16 with React 19, TypeScript 5, and Tailwind CSS 4. It provides server listing and search with grid and list views, detailed server pages showing tool definitions, security report summaries, repository metadata, and pricing. The interface also provides Firebase-authenticated user profiles, a guided publish workflow modal, a Razorpay-integrated paywall for premium servers, and an interactive playground for live server testing. The frontend communicates with the Go API for all data operations and with Firebase directly for authentication state management.

E. Infrastructure

All AWS infrastructure is provisioned and managed using Terraform with a modular structure. The S3 module creates the registry bucket with appropriate access policies. The IAM module creates the Lambda execution role with `s3:GetObject`, `logs:CreateLogGroup`, `logs:CreateLogStream`, and `logs:PutLogEvents` permissions. The Lambda module packages and deploys the handler with environment variables and configures a Function URL for direct invocation. The WebSocket module creates the API Gateway WebSocket API, defines the three route

integrations, creates a Lambda integration with the handler, and deploys to a named stage. Outputs from each module are passed as inputs to dependent modules, and the final WebSocket URL and S3 bucket name are output for use in the application environment configuration.

V. RESULTS AND EVALUATION

A. Security Pipeline Evaluation

The five-stage security pipeline was evaluated against a set of ten MCP server repositories seeded with known vulnerability classes: hardcoded API keys and database credentials, dependencies with known CVEs at varying severity levels, Python-specific dangerous patterns such as `eval()` and `subprocess` with shell injection risks, and tool descriptions containing prompt injection payloads. Each vulnerability class was represented by at least one seeded instance per repository. The pipeline correctly identified and flagged every seeded finding. SonarCloud detected all code quality and injection-risk patterns. Snyk flagged all CVE-matched dependencies. GitGuardian identified all hardcoded credential patterns. Bandit caught all Python-specific dangerous patterns. No false-positive rejections were observed on clean repositories.

The tool discovery stage successfully extracted tool definitions from all test repositories, including servers using both decorator-based and registration-based MCP tool definition patterns.

B. Execution Performance

Lambda cold start performance was measured as the total time from WebSocket `$connect` event to the first valid JSON-RPC response from the bootstrapped MCP server, across server repositories of varying size and dependency complexity. Table V summarizes the results.

TABLE V
LAMBDA COLD START LATENCY BY REPOSITORY PROFILE

Repository Profile	Deps	Observed Latency
Minimal Python (no external deps)	0	~8 s
Light Python (stdlib + 1–3 pkgs)	1–3	~25 s
Medium Python (several packages)	5–15	~55 s
Heavy Python (large frameworks)	15+	~120 s

These measurements confirm that cold start latency is dominated by dependency installation time. Subsequent messages within the same WebSocket session are forwarded with sub-100 ms overhead, as the MCP process remains alive for the connection duration. The Lambda container reuse behavior reduces cold starts for high-frequency servers.

C. Registry and API Performance

The S3-backed flat-file registry demonstrated consistent read performance for list and individual retrieval operations. The Go API returned full server lists with security reports in under 200 ms for registries of up to 500 servers in testing. A DynamoDB-backed equivalent with the same metadata

schema returned responses in a comparable range but introduced additional connection management overhead and per-read capacity unit cost. The flat-file design eliminates query planning overhead and enables direct S3 `GetObject` calls for individual server lookups, producing predictable latency regardless of registry size.

D. End-to-End Validation

The complete publish-to-execution cycle was validated across ten MCP server repositories spanning Python REST API wrappers, file system tools, database query tools, and web scraping servers. All servers that cleared the five-stage pipeline were confirmed to be discoverable through the API immediately after publication, with tool lists returned by the Lambda executor at runtime matching the tool definitions extracted during publish-time tool discovery. Client configuration files generated by the CLI for all supported AI clients (Claude Desktop, Cursor, VSCode, Windsurf, and ChatGPT) were confirmed to correctly route JSON-RPC traffic through the local proxy to the Lambda execution layer.

VI. CONCLUSION AND FUTURE WORK

SuperBox addresses a critical infrastructure gap in the MCP ecosystem by providing an integrated platform for centralized discovery, automated security validation, and sandboxed execution of MCP servers. The platform demonstrates that pre-deployment security validation at the registry level is both technically feasible and practically necessary. A serverless execution model built on AWS Lambda and WebSocket API Gateway provides the session isolation and scalability required for safe MCP server integration in production environments.

The primary contribution of this work is the shift of the security boundary upstream to the point of publication rather than relying on runtime defenses after deployment. By enforcing the five-stage pipeline as a precondition for registry entry, the trust boundary is established at publication time, complementing the runtime defenses proposed in related work [3], [9]. The S3-backed flat-file registry design provides a portable, auditable, and operationally simple foundation for the marketplace that scales without a dedicated database tier.

The current implementation carries several limitations that bound the scope of these results. Execution support is restricted to Python servers; Node.js and other runtimes are not yet handled, excluding a significant portion of the MCP ecosystem. Cold start latency for dependency-heavy servers can exceed two minutes, making the platform unsuitable for latency-sensitive workloads without warm container strategies. The tool discovery stage relies on regex pattern matching, which may miss dynamically registered tools or tools defined outside standard decorator patterns. The security pipeline gates publication but does not continuously monitor deployed servers for newly disclosed CVEs in their dependencies.

Future work will target four directions: extending Lambda execution to Node.js runtimes to cover the JavaScript portion of the MCP ecosystem; automated repository webhook re-scanning to keep security reports current without manual re-

publication; MCP server versioning with rollback capability to address the rug pull threat class identified in the literature [3]; and community-contributed ratings and download metrics to add social trust signals alongside the automated pipeline results.

ACKNOWLEDGMENT

The authors thank Ms. Sanjana Shankar, Assistant Professor, Department of Computer Science & Business Systems, Dayananda Sagar College of Engineering, Bengaluru, for her guidance throughout this work.

REFERENCES

- [1] Anthropic, “Model Context Protocol Specification,” Nov. 2024. [Online]. Available: <https://modelcontextprotocol.io/specification>
- [2] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, J. E. Gonzalez, R. A. Popa, I. Stoica, and D. A. Patterson, “Cloud Programming Simplified: A Berkeley View on Serverless Computing,” *Tech. Rep. UCB/EECS-2019-3*, EECS Dept., Univ. of California, Berkeley, Feb. 2019. [Online]. Available: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-3.html>
- [3] S. Jamshidi, K. W. Nafi, A. M. Dakhel, N. Shahabi, F. Khomh, and N. Ezzati-Jivan, “Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks,” *arXiv preprint*, arXiv:2512.06556 [cs.CR], Dec. 2025. [Online]. Available: <https://arxiv.org/abs/2512.06556>
- [4] H. Guo, Y. Hao, Y. Zhang, M. Xu, P. Lv, J. Chen *et al.*, “A Measurement Study of Model Context Protocol Ecosystem,” *arXiv preprint*, arXiv:2509.25292 [cs.CY], Sep. 2025. [Online]. Available: <https://arxiv.org/abs/2509.25292>
- [5] S. Zhao, Q. Hou, Z. Zhan, Y. Wang, Y. Xie, Y. Guo *et al.*, “Mind Your Server: A Systematic Study of Parasitic Toolchain Attacks on the MCP Ecosystem,” *arXiv preprint*, arXiv:2509.06572 [cs.CR], Sep. 2025. [Online]. Available: <https://arxiv.org/abs/2509.06572>
- [6] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers,” *arXiv preprint*, arXiv:2506.13538 [cs.SE], Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.13538>
- [7] H. Song, Y. Shen, W. Luo, L. Guo, T. Chen, J. Wang *et al.*, “Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol (MCP) Ecosystem,” *arXiv preprint*, arXiv:2506.02040 [cs.CR], Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.02040>
- [8] Z. Wang, R. Zhang, Y. Liu, W. Fan, W. Jiang, Q. Zhao *et al.*, “MPMA: Preference Manipulation Attack Against Model Context Protocol,” *arXiv preprint*, arXiv:2505.11154 [cs.CR], May 2025. [Online]. Available: <https://arxiv.org/abs/2505.11154>
- [9] V. S. Narajala, K. Huang, and I. Habler, “Securing GenAI Multi-Agent Systems Against Tool Squatting: A Zero Trust Registry-Based Approach,” *arXiv preprint*, arXiv:2504.19951 [cs.CR], Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2504.19951>
- [10] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions,” *arXiv preprint*, arXiv:2503.23278 [cs.CR], Mar. 2025. [Online]. Available: <https://arxiv.org/abs/2503.23278>